

HDOC DAY-30

ARRAY OPERATIONS USING MENU-DRIVEN C PROGRAM

Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Subject	C Programming
Activity	Bubble Sort & Binary Search, Insertion & Deletion, Right Rotation

Problem Statement

Write a C program to perform the following menu-driven operations on a given array: (1) sort the elements using Bubble Sort and perform Binary Search, (2) perform insertion and deletion at a given valid position, (3) rotate the array elements to the right by k positions, and (4) exit.

1. Algorithm

1. Start the program.
2. Read the number of elements and the array elements.
3. Display the menu repeatedly until the user chooses Exit.
4. If choice = 1:
 - a. Sort the array in ascending order using Bubble Sort.
 - b. Display the sorted array.
 - c. Read the search element and perform Binary Search.
 - d. Display whether the element is found and its position.
5. If choice = 2:
 - a. Ask whether the user wants insertion or deletion.
 - b. For insertion, read a valid position and value, shift elements right, and insert.
 - c. For deletion, read a valid position, shift elements left, and reduce the size.
6. If choice = 3:
 - a. Read k.
 - b. Use $k = k \% n$.
 - c. Rotate the array to the right by k positions.
7. If choice = 4, terminate the program.
8. For any other choice, display an invalid-choice message.
9. Stop.

2. Test Case Table

Test	Input / Operation	Expected Result
1	Array: 5 2 9 1 7; Sort; Search 7	Sorted: 1 2 5 7 9; 7 found
2	Insert 25 at position 3 in 10 20 30 40	10 20 25 30 40
3	Delete position 2 from 10 20 30 40	10 30 40
4	Rotate 1 2 3 4 5 right by k=2	4 5 1 2 3
5	Search 8 in sorted 1 3 5 7 9	Element not found

3. C Program

```
#include <stdio.h>

#define MAX 100

void display(int a[], int n) {
    int i;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}

void bubbleSort(int a[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (a[mid] == key) return mid;
        else if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

void insertElement(int a[], int *n) {
    int pos, value, i;
    if (*n >= MAX) { printf("Array is full.\n"); return; }
    printf("Enter position (1 to %d): ", *n + 1);
    scanf("%d", &pos);
    if (pos < 1 || pos > *n + 1) { printf("Invalid position.\n"); return; }
    printf("Enter value: ");
    scanf("%d", &value);
    for (i = *n; i >= pos; i--) a[i] = a[i - 1];
    a[pos - 1] = value;
    (*n)++;
    printf("Array after insertion: "); display(a, *n);
}

void deleteElement(int a[], int *n) {
    int pos, i;
    if (*n == 0) { printf("Array is empty.\n"); return; }
    printf("Enter position (1 to %d): ", *n);
    scanf("%d", &pos);
    if (pos < 1 || pos > *n) { printf("Invalid position.\n"); return; }
    for (i = pos - 1; i < *n - 1; i++) a[i] = a[i + 1];
    (*n)--;
    printf("Array after deletion: "); display(a, *n);
}

void rotateRight(int a[], int n, int k) {
    int temp[MAX], i;
    if (n == 0) return;
    k = k % n;
    for (i = 0; i < n; i++) temp[(i + k) % n] = a[i];
    for (i = 0; i < n; i++) a[i] = temp[i];
}

int main() {
    int a[MAX], n, i, choice, subChoice, key, result, k;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    if (n < 1 || n > MAX) { printf("Invalid array size.\n"); return 0; }
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    do {
```

```

printf("\n--- MENU ---\n");
printf("1. Bubble Sort and Binary Search\n");
printf("2. Insertion and Deletion\n");
printf("3. Right Rotation by k positions\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        bubbleSort(a, n);
        printf("Sorted array: "); display(a, n);
        printf("Enter element to search: "); scanf("%d", &key);
        result = binarySearch(a, n, key);
        if (result != -1) printf("Element found at position %d.\n", result + 1);
        else printf("Element not found.\n");
        break;
    case 2:
        printf("1. Insertion\n2. Deletion\nEnter choice: ");
        scanf("%d", &subChoice);
        if (subChoice == 1) insertElement(a, &n);
        else if (subChoice == 2) deleteElement(a, &n);
        else printf("Invalid sub-choice.\n");
        break;
    case 3:
        printf("Enter k: "); scanf("%d", &k);
        if (k < 0) printf("Enter a non-negative value of k.\n");
        else { rotateRight(a, n, k); printf("Array after rotation: "); display(a, n); }
        break;
    case 4:
        printf("Program terminated.\n");
        break;
    default:
        printf("Invalid choice.\n");
}
} while (choice != 4);
return 0;
}

```

4. Execution Screenshot

Program execution in Terminal - HDOC Day-30

```
dhruvpandey@DHUVs-MacBook-Air ~ % ./day30
```

```
===== ARRAY OPERATIONS =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate Array Right by k Positions
4. Exit
```

```
=====
Enter your choice: 1
Enter number of elements: 2
Enter 2 elements:
3
4
Sorted array: 3 4
Enter element to search: 3
Element 3 found at position 1.
```

```
===== ARRAY OPERATIONS =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate Array Right by k Positions
4. Exit
```

```
=====
Enter your choice: 2
Enter number of elements: 2
Enter 2 elements:
5
6
```

```
1. Insertion
2. Deletion
Enter your choice: 1
Enter position for insertion: 1
Enter element to insert: 1
Array after insertion: 1 5 6
```

```
===== ARRAY OPERATIONS =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate Array Right by k Positions
4. Exit
```

```
=====
Enter your choice: 3
Enter number of elements: 2
Enter 2 elements:
3
2
Enter number of positions to rotate: 1
Array after right rotation: 2 3
```

```
===== ARRAY OPERATIONS =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate Array Right by k Positions
4. Exit
```

```
=====
Enter your choice: █
```

4. Sample Execution

```
Enter number of elements: 5
Enter 5 elements: 5 2 9 1 7

--- MENU ---
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Right Rotation by k positions
4. Exit
Enter your choice: 1
Sorted array: 1 2 5 7 9
Enter element to search: 7
Element found at position 4.

Enter your choice: 2
1. Insertion
2. Deletion
Enter choice: 1
Enter position (1 to 6): 3
Enter value: 25
Array after insertion: 1 2 25 5 7 9

Enter your choice: 3
Enter k: 2
Array after rotation: 7 9 1 2 25 5

Enter your choice: 4
Program terminated.
```

5. Result

The menu-driven C program was implemented successfully. It performs Bubble Sort followed by Binary Search, insertion and deletion at valid positions, and right rotation of the array by k positions.